

Rising Waters: A Machine Learning Approach to Flood Prediction

Project Description:

Rising Waters is a machine learning powered flood early-warning system built for Indian meteorological and disaster-management use cases. The platform includes a Flood Risk Classifier, which analyses ten weather and seasonal rainfall readings to predict whether a flood is likely; a Probability Engine, which converts the model's raw output into a confidence percentage; and a Safety Recommendation panel, which surfaces context-specific precautions or reassurance depending on the predicted outcome.

The underlying model is trained in a Jupyter notebook on 115 labelled weather/flood records, comparing Decision Tree, Random Forest, K-Nearest Neighbors, and XGBoost classifiers, and saving the best-performing model together with its StandardScaler as a single joblib bundle (model/floods.save). Built with Flask and deployed publicly on Render, the platform ensures a fast, glassmorphism-styled experience. With the model and scaler bundled together and the feature list generated automatically at startup, Rising Waters empowers meteorologists, disaster-relief coordinators, and government analysts to assess flood risk with confidence.

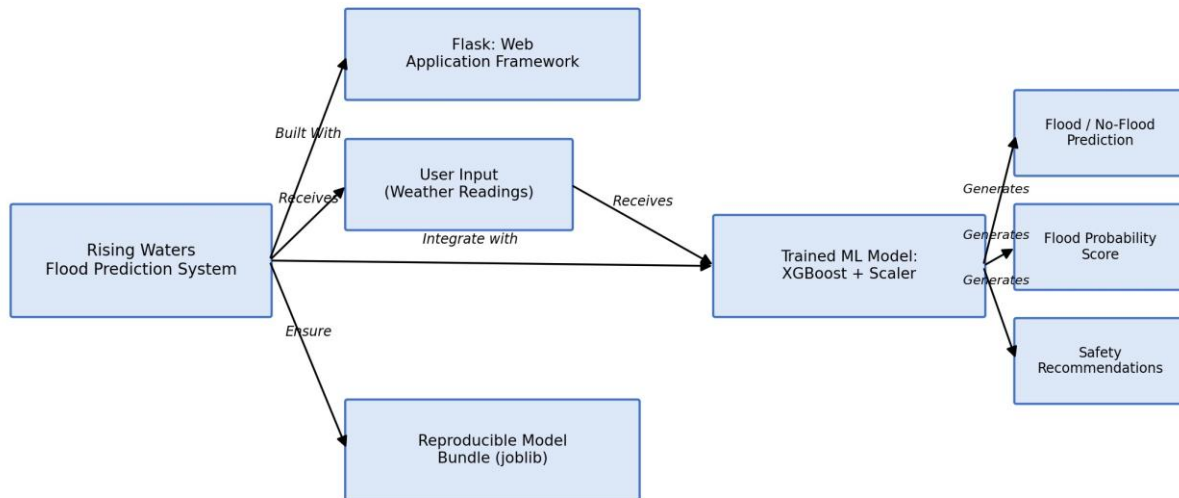
Scenarios

Scenario 1: A meteorologist enters the current temperature, humidity, cloud cover, and monsoon-season rainfall totals for a subdivision experiencing unusually heavy Jun–Sep rain. The system classifies the reading as “Flood Likely,” reports a high confidence percentage, and lists an emergency checklist — activating alarms, monitoring river levels, and preparing shelters.

Scenario 2: A disaster-relief coordinator checks a subdivision with moderate, near-average seasonal rainfall. Rising Waters predicts “No Flood Likely” with a low risk percentage, allowing the coordinator to keep resources allocated to higher-risk districts instead.

Scenario 3: A government analyst retrains the model after a new season's data is added to flood dataset.xlsx, re-runs the notebook, and confirms that XGBoost still matches or exceeds Random Forest and Decision Tree accuracy before the updated model/floods.save is redeployed to the live Flask app — no code changes required unless the underlying feature columns change.

Technical Architecture:



Description: Rising Waters utilizes a modular three-tier architecture to deliver fast, ML-driven flood classification. The frontend provides a responsive glassmorphism interface built with HTML and CSS, while the Flask-based backend validates the ten submitted readings, orders them to match the model's expected feature order, and scales them with the saved StandardScaler. It then passes the scaled inputs to the trained classifier (XGBoost, selected for deployment) to produce a prediction and probability score, with the model and scaler both loaded once at startup from model/floods.save and served publicly via Render.

(Note: Rising Waters does not use a relational database — all persisted state is the trained model bundle itself, so no ER diagram applies to this project.)

Pre-requisites:

- **Flask Framework Knowledge:** Flask Documentation
- **scikit-learn & XGBoost Familiarity:** scikit-learn Documentation / XGBoost Documentation
- **Pandas & NumPy Proficiency:** Pandas Documentation
- **HTML, CSS, and JavaScript Skills:** W3Schools HTML/CSS/JavaScript Tutorials
- **Python Programming Proficiency:** Python Documentation
- **Version Control with Git:** Git Documentation
- **Jupyter Notebook Setup:** Jupyter Installation Guide
- **Docker for Containerized Deployment:** Docker Documentation
- **Render for Public Deployment:** Render Documentation

Project Workflow:

Milestone 1: Data Collection and Environment Setup

- **Activity 1.1:** Collect the rainfall and flood datasets and load them for exploratory analysis.
- **Activity 1.2:** Research and compare candidate classifiers (Decision Tree, Random Forest, KNN, XGBoost) for the flood-prediction task.

- **Activity 1.3:** Define the application architecture, detailing interactions between the frontend, Flask backend, and the trained model bundle.
- **Activity 1.4:** Set up the development environment, installing the libraries needed for the notebook and the Flask app.

Milestone 2: Model Training and Evaluation

- **Activity 2.1:** Pre-process the flood dataset (scaling, train/test split) and train all four candidate classifiers.
- **Activity 2.2:** Evaluate each model's test accuracy, compare results, and select the best-performing model for deployment.
- **Activity 2.3:** Save the selected model together with its StandardScaler and feature list as a single joblib bundle.

Milestone 3: Flask App Development (app.py)

- **Activity 3.1:** Write the main application logic in app.py, loading the model bundle and establishing the home, predict, and result routes.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the glassmorphism user interface using HTML, CSS, and Jinja templates, ensuring a responsive and intuitive layout.
- **Activity 4.2:** Create dynamic templates that render the prediction form from the model's own feature list and display flood/no-flood results.

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment by configuring gunicorn, the Procfile, and requirements.
- **Activity 5.2:** Deploy the application publicly on Render (with Docker and Cloud Foundry manifests provided as alternatives).

Milestone 6: Conclusion

Milestone 1: Data Collection and Environment Setup

In this milestone, we focus on gathering the rainfall and flood datasets and choosing the right set of classifiers for Rising Waters. This involves reviewing what each dataset contains, exploring the candidate scikit-learn / XGBoost models, and setting up a development environment that supports both the notebook and the Flask app.

Activity 1.1: Collect and Inspect the Datasets

Two datasets are used, since together they cover both the broad historical picture and the ground-truth flood outcomes:

- **rainfall in india 1901-2015.xlsx** — monthly and seasonal rainfall totals for 36 Indian subdivisions across 115 years (4,116 rows), used for the broad exploratory analysis.

- **flood dataset.xlsx** — 115 records with temperature, humidity, cloud cover, and seasonal rainfall totals (ANNUAL, Jan-Feb, Mar-May, Jun-Sep, Oct-Dec, avgjune, sub), plus a binary flood label. This is the dataset the four classifiers are actually trained and evaluated on.

Activity 1.2: Research and Select the Candidate Models

Here are the things that needed to be researched to select the best models for the project:

- **Understand the Project Requirements:** Review the specific needs of Rising Waters — a binary flood / no-flood classification from ten numeric weather and rainfall readings.
- **Explore scikit-learn's Model Documentation:** Examine Decision Tree, Random Forest, and K-Nearest Neighbors, including their strengths on small tabular datasets.
- **Evaluate XGBoost:** Compare a boosted-tree ensemble against the scikit-learn models for its tendency to generalize better on unseen data.
- **Select the Optimal Model:** Choose **XGBoost** for deployment, since it tends to generalize better than a single Decision Tree even when test accuracy ties on this small (115-record) dataset.

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture, including the frontend (HTML, CSS), the Flask backend, and the trained model bundle.
- **Detail Frontend Functionality:** Outline how users interact with the application — entering ten weather/rainfall readings across three field groups (Atmospheric Readings, Seasonal Rainfall, Derived Rainfall Indicators).
- **Outline Backend Responsibilities:** Specify how the Flask backend handles incoming POST requests to /predict, validates and orders user input, scales it, and calls the model.
- **Describe Model Integration Points:** Define how app.py loads MODEL, SCALER, and FEATURE_NAMES once at startup from model/floods.save, and how predict() builds a DataFrame in the model's expected column order before scaling and predicting.

Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3.11+ is installed along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment to isolate project dependencies:

```
python -m venv venv
venv\Scripts\activate (Windows) source
venv/bin/activate (macOS / Linux) pip
install -r requirements.txt
```

- **Install the notebook + app dependencies:** numpy, pandas, matplotlib, seaborn, scikit-learn, xgboost, flask, joblib, openpyxl, gunicorn (all listed in requirements.txt).
- **Set Up Application Structure:** The project directory separates data/, notebook/, model/, and app/ (with templates/ and static/css/ inside app/) — see the folder structure in the Project Executable Files document.

Milestone 2: Model Training and Evaluation

Milestone 2 covers pre-processing the flood dataset, training all four candidate classifiers, comparing their accuracy, and saving the selected model for deployment.

Activity 2.1: Pre-process the Data and Train the Classifiers

- Load flood dataset.xlsx and separate the ten feature columns from the binary flood target.
- Fit a StandardScaler on the training split and apply it to both the training and test features.
- Train Decision Tree, Random Forest, K-Nearest Neighbors, and XGBoost (falling back to GradientBoostingClassifier automatically if xgboost isn't installed) on the same split.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
try:
    from xgboost import XGBClassifier as BoostedModel
except ImportError:
    from sklearn.ensemble import GradientBoostingClassifier as BoostedModel

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

scaler = StandardScaler().fit(X_train)
X_train_s, X_test_s = scaler.transform(X_train), scaler.transform(X_test)
```

Activity 2.2: Evaluate and Compare Model Accuracy

Results on this dataset:

Model	Test Accuracy
Decision Tree	95.65%
Random Forest	95.65%
K-Nearest Neighbors	86.96%
XGBoost (selected for deployment)	95.65%

With only 115 labelled records, several models land on the same accuracy on this particular train/test split — when that happens, XGBoost is preferred for deployment since it tends to generalize better than a single Decision Tree on unseen data.

Activity 2.3: Save the Model Bundle

The selected model, its scaler, the feature name order, the model's display name, and its accuracy are bundled together with joblib so the Flask app can load everything it needs from one file:

```
import joblib

joblib.dump({
    "model": best_model,
    "scaler": scaler,
    "feature_names": list(X.columns),
    "model_name": "XGBoost",
    "accuracy": test_accuracy,
}, "model/floods.save")
```

Milestone 3: Flask App Development (app.py)

Milestone 3 focuses on writing app.py, which loads the trained model bundle once at startup and exposes the home, prediction-form, and result routes.

Activity 3.1: Load the Model Bundle at Startup

```
_bundle = joblib.load(MODEL_PATH)
MODEL = _bundle["model"]
SCALER = _bundle["scaler"]
FEATURE_NAMES = _bundle["feature_names"]
MODEL_NAME = _bundle["model_name"]
MODEL_ACCURACY = _bundle["accuracy"]
```

Activity 3.2: Define the Home and Predict Routes

```
@app.route("/")
def home():
    return render_template("home.html", model_name=MODEL_NAME,
        accuracy=round(MODEL_ACCURACY * 100, 2), n_features=len(FEATURE_NAMES))

@app.route("/predict", methods=["GET", "POST"])
def predict():
    if request.method == "GET":
        return render_template("predict.html", groups=FIELD_GROUPS,
            meta=FIELD_META, errors=None, values={})

    values, errors = {}, {}
    for f in FEATURE_NAMES:
        raw = request.form.get(f, "").strip()
        try:
            values[f] = float(raw)
        except ValueError:
            errors[f] = "Enter a valid number."
```

Activity 3.3: Score the Input and Render the Result

```
ordered = pd.DataFrame([[values[f] for f in FEATURE_NAMES]], columns=FEATURE_NAMES)
scaled = SCALER.transform(ordered) pred = int(MODEL.predict(scaled)[0])

try:
    proba = MODEL.predict_proba(scaled)[0]
    flood_probability = float(proba[1]) * 100
except AttributeError: flood_probability =
100.0 if pred == 1 else 0.0

template = "flood_result.html" if pred == 1 else "no_flood_result.html"
return render_template(template, probability=round(flood_probability, 1),
values=values, meta=FIELD_META, model_name=MODEL_NAME)
```

This keeps the app fully data-driven: retraining the notebook and overwriting model/floods.save is enough to deploy an updated model, with no code changes needed unless the underlying feature columns themselves change.

Milestone 4: Frontend Development

Milestone 4 focuses on developing a clear, trustworthy interface for Rising Waters. This involves designing a glassmorphism-style layout with responsive HTML and CSS, and building Jinja templates that render the prediction form directly from the model's own feature list.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base Template:

- Develop a shared base.html layout with the Rising Waters brand, a sticky navigation bar (Home, Predict Flood Risk, How It Works, Contact), and a footer showing the live model name and accuracy.
- Use semantic HTML and Font Awesome icons for section headers, with Poppins for display text and Inter for body copy.

Design a Responsive Layout Using CSS:

- Create static/css/style.css implementing a light glassmorphism aesthetic (frosted .glass-card panels, blue/navy accent palette, 16px rounded corners).
- Use CSS grid to lay the prediction form out in two columns per field group, and to place the result page's image and details side by side.
- Add media queries so the layout collapses to a single column on tablet and mobile widths.

Create Separate UI Components for Each Output Type:

- **Field Groups:** the ten inputs are organised into Atmospheric Readings, Seasonal Rainfall, and Derived Rainfall Indicators, each with a label, unit suffix, and helper text pulled from FIELD_META.
- **Confidence Card:** a glass card showing the flood probability as a large percentage plus an animated progress bar.
- **Action Lists:** a danger-styled checklist on the Flood Likely page and a safe-styled recommendation list on the No Flood Likely page.

Activity 4.2: Rendering the Prediction Form Dynamically

```
FIELD_GROUPS = [  
    ("Atmospheric Readings", ["Temp", "Humidity", "Cloud Cover"]),  
    ("Seasonal Rainfall", ["Jan-Feb", "Mar-May", "Jun-Sep", "Oct-Dec"]), ("Derived  
Rainfall Indicators", ["ANNUAL", "avgjune", "sub"]),  
]
```

The predict.html template loops over FIELD_GROUPS and, for each field, looks up its label, unit, min/max, and helper text from FIELD_META — so the form always matches whatever ten features the currently loaded model actually expects.

Milestone 5: Deployment

In Milestone 5, the focus is on preparing Rising Waters for production and deploying it publicly on Render, with Docker and Cloud Foundry manifests included as alternative deployment paths.

Activity 5.1: Preparing the Application for Deployment

Configure gunicorn via the Procfile:

```
web: gunicorn --chdir app --bind 0.0.0.0:$PORT app:app
```

Or containerize with the included Dockerfile:

```
FROM python:3.11-slim  
WORKDIR /srv  
COPY app/requirements.txt ./requirements.txt RUN  
pip install --no-cache-dir -r requirements.txt  
COPY model/ ./model/  
COPY app/ ./app/  
WORKDIR /srv/app  
EXPOSE 5000  
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "app:app"]
```

In every case, the model/ folder must be deployed alongside app/, since the app loads ../model/floods.save relative to app.py.

Activity 5.2: Deploy Publicly on Render

- Push the repository to GitHub: <https://github.com/lakshmilathish/rising-waters>
- Create a new Web Service on Render and connect it to the GitHub repository.
- Set the build command to `pip install -r requirements.txt` and the start command to the Procfile's gunicorn command.
- Render builds and deploys automatically on every push, exposing the app at a public *.onrender.com URL. The live deployment is available at: <https://rising-waters-gx15.onrender.com>

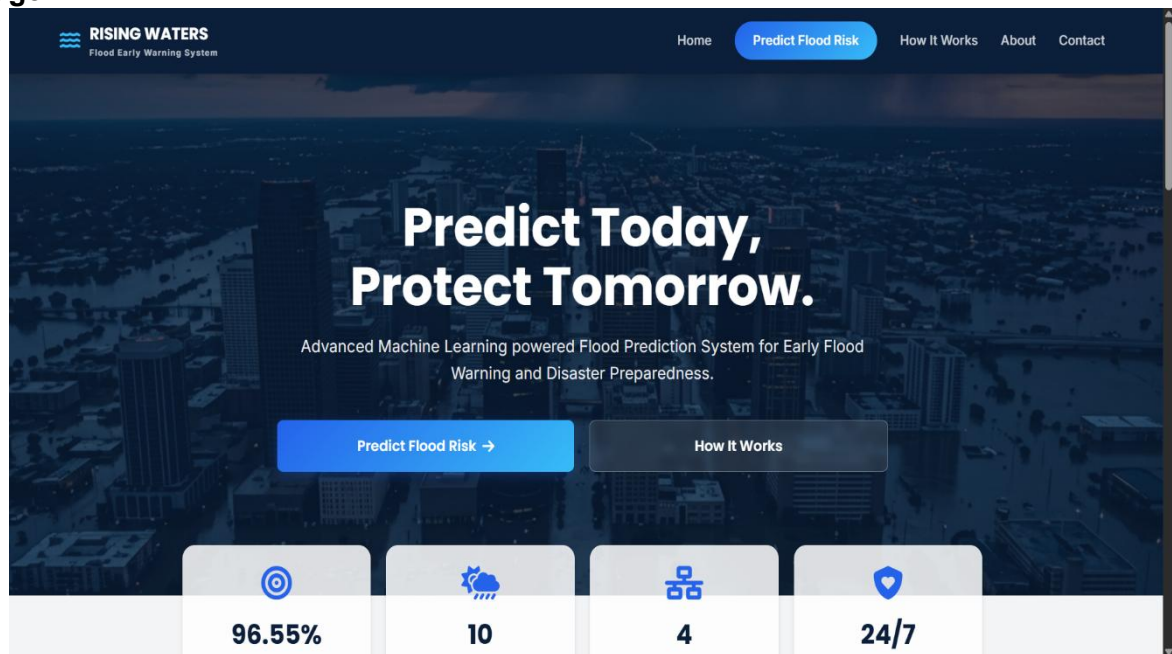
Important Notes:

- Render's free tier can spin down when idle, so the first request after inactivity may take a few seconds longer while the service wakes up.
- Retraining the notebook overwrites model/floods.save locally — remember to commit and push the updated file for it to reach the deployed app.

- No .env or database configuration is required; the only external file the app depends on at runtime is model/floods.save.

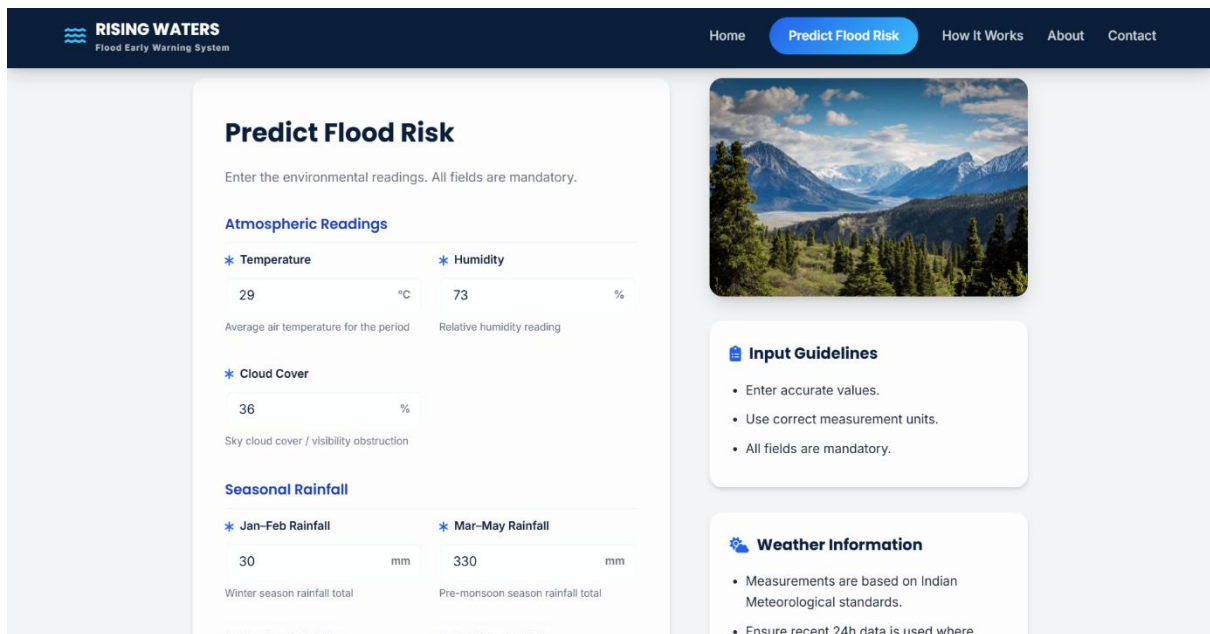
Exploring the Website's Web Pages:

Home Page:



Description: The home page introduces Rising Waters with a full-width hero banner, live model accuracy and feature-count stat cards, a grid of feature highlights (AI-powered prediction, environmental monitoring, early warning, disaster preparedness), and a four-step “How It Works” timeline.

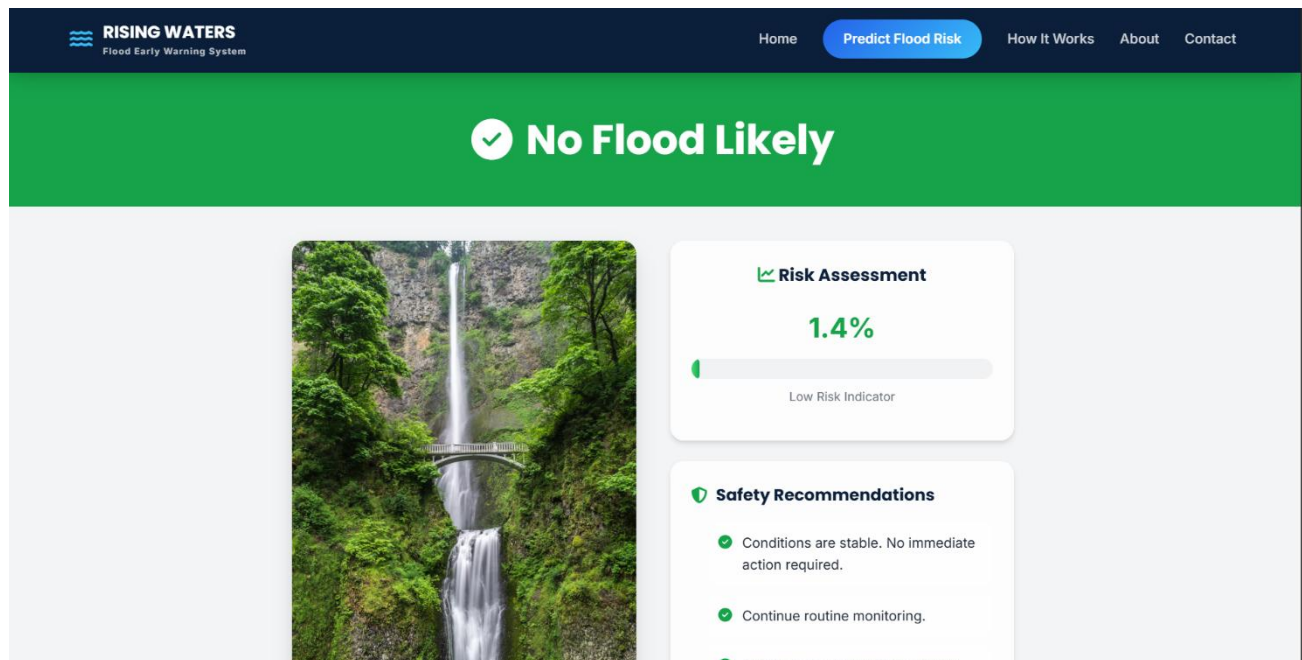
Prediction Console:



The screenshot shows the 'Predict Flood Risk' form in the Rising Waters application. The form is titled 'Predict Flood Risk' and includes a note: 'Enter the environmental readings. All fields are mandatory.' The form is divided into three sections: 'Atmospheric Readings', 'Seasonal Rainfall', and 'Weather Information'. The 'Atmospheric Readings' section contains fields for Temperature (29 °C), Humidity (73 %), and Cloud Cover (36 %). The 'Seasonal Rainfall' section contains fields for Jan-Feb Rainfall (30 mm) and Mar-May Rainfall (330 mm). The 'Weather Information' section includes a list of guidelines: 'Enter accurate values.', 'Use correct measurement units.', and 'All fields are mandatory.' The form also includes a 'Predict Flood Risk' button and a 'How It Works' button.

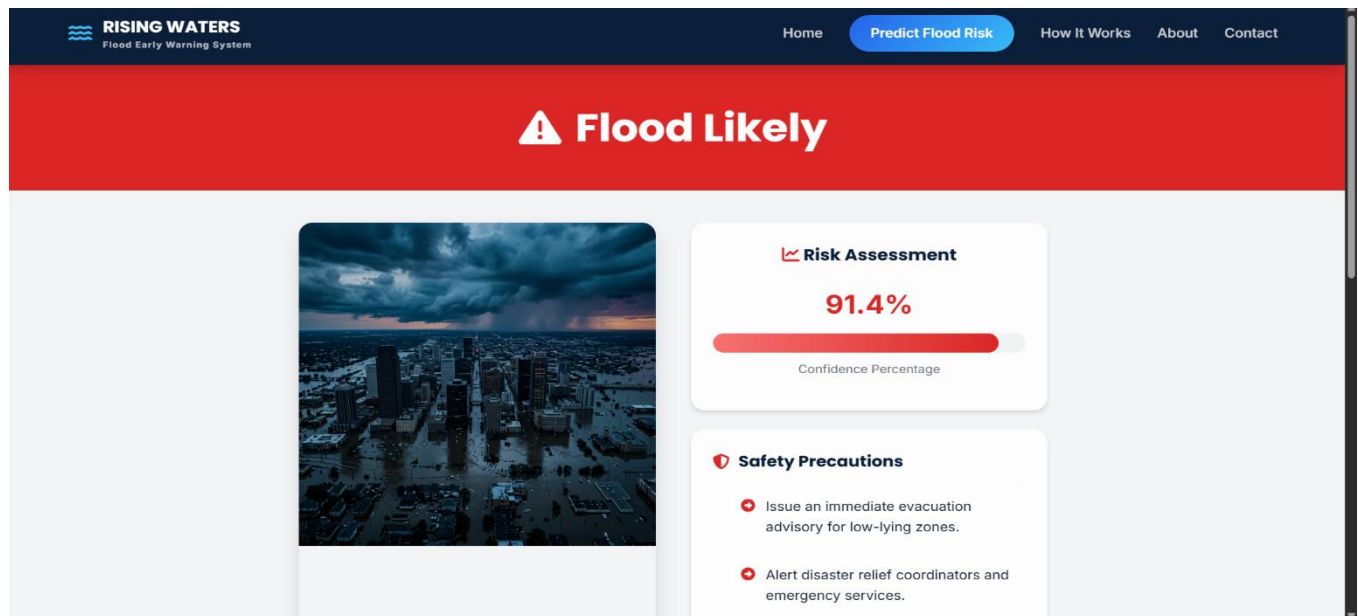
Description: A glass-styled card containing the ten-field prediction form, grouped into Atmospheric Readings, Seasonal Rainfall, and Derived Rainfall Indicators. Each field shows its unit, a helper caption, and inline validation errors if a value is missing or invalid. A sidebar offers input guidelines and general weather-data notes.

Flood Likely Result Page:



Description: Shown automatically when the model predicts a flood. Displays the predicted flood probability on an animated progress bar, a danger-styled safety-precaution list (evacuation advisories, alerting coordinators, pre-positioning resources), and an emergency checklist.

No Flood Likely Result Page:



Description: Shown automatically when the model predicts no flood. Displays the low risk percentage and a safe-styled recommendation list confirming stable conditions and routine monitoring, letting resources stay allocated to higher-risk districts.

Conclusion:

Rising Waters is a machine-learning-powered flood early-warning platform built with Flask and styled with a light glassmorphism UI. It trains and compares Decision Tree, Random Forest, K-Nearest Neighbors, and XGBoost classifiers on Indian rainfall and flood data, deploying the strongest model — XGBoost — as a single joblib bundle that the Flask app loads at startup. The project, which spanned dataset collection, model training and evaluation, Flask backend development, glassmorphism frontend design, and public deployment on Render, highlights how a small, well-curated dataset and a structured pipeline can still produce a genuinely useful early-warning tool for meteorologists, disaster-relief coordinators, and government analysts.

Looking ahead, Rising Waters offers clear opportunities for expansion, such as collecting more labelled flood records to reduce sensitivity to the train/test split, adding a database to log and review past predictions, integrating live weather-API data instead of manual entry, and hosting the hero and result imagery locally so the app no longer depends on external image URLs. By continuing to refine the underlying model and broadening its data sources, the platform is well-positioned to grow from a demonstration project into a genuinely operational flood early-warning tool.